

```
In [2]: import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import transforms
from facenet_pytorch import InceptionResnetV1, MTCNN
from PIL import Image, ImageFilter, ImageEnhance
import numpy as np
from facenet_pytorch.models.inception_resnet_v1 import InceptionResnetV1
from torch import Tensor
import random
from collections import defaultdict
from typing import List, Tuple, Set, Literal
```

```
In [3]: device: Literal['cuda'] | Literal['cpu'] = 'cuda' if torch.cuda.is_available() else
print(f"Using device: {device}")
```

Using device: cuda

```
In [4]: image_name_to_person_id: dict[str, int] = {}
person_id_to_image_names: dict[int, list[str]] = defaultdict(list)

with open("identity_CelebA.txt", "r") as lines:
    for line in lines:
        image_name, person_id_str = line.split(" ")
        person_id = int(person_id_str.strip())
        image_name_to_person_id[image_name.strip()] = person_id
        person_id_to_image_names[person_id].append(image_name.strip())

person_ids_with_multiple_images: list[int] = [pid for pid, images in person_id_to_image_names.items() if len(images) > 1]
all_image_names = list(image_name_to_person_id.keys())

DATA_DIR = "img_align_celeba/"
```

```
In [5]: def get_balanced_training_data(
    total_pairs: int,
    exclude_images: Set[str] = set()
) -> List[Tuple[str, str, int]]:
    """
    Generates a balanced dataset of image pairs for training or validation.
    Aims for a 50/50 split between "same person" and "different people" pairs.
    Ensures that no images from the `exclude_images` set are used.
    """
    data = set()
    used_pairs = set()

    # --- 1. Generate "Same Person" (positive) pairs ---
    num_same_pairs = total_pairs // 2

    available_persons = [
        pid for pid in person_ids_with_multiple_images
        if not all(img in exclude_images for img in person_id_to_image_names[pid])
    ]
    if not available_persons:
        raise ValueError("Not enough available persons to generate 'same' pairs")

    while len(data) < num_same_pairs:
        person_id = random.choice(available_persons)
        possible_images = [img for img in person_id_to_image_names[person_id] if img not in exclude_images]
```

```

        if len(possible_images) < 2:
            continue

        img1, img2 = random.sample(possible_images, 2)

        pair = tuple(sorted((img1, img2)))
        if pair not in used_pairs:
            data.add((img1, img2, 1)) # Label 1 for "same"
            used_pairs.add(pair)

# --- 2. Generate "Different People" (negative) pairs ---
available_images = [img for img in all_image_names if img not in exclude_images]
if len(available_images) < 2:
    raise ValueError("Not enough available images to generate 'different' pairs")

while len(data) < total_pairs:
    img1, img2 = random.sample(available_images, 2)

    if image_name_to_person_id[img1] != image_name_to_person_id[img2]:
        pair = tuple(sorted((img1, img2)))
        if pair not in used_pairs:
            data.add((img1, img2, 0)) # Label 0 for "different"
            used_pairs.add(pair)

final_data = list(data)
random.shuffle(final_data)
return final_data

```

```

In [6]: class FaceVerificationMLP(nn.Module):
        '''The MLP model that classifies the difference vector between two face embeddings'''
        def __init__(self, input_dim=512):
            super(FaceVerificationMLP, self).__init__()
            self.model = nn.Sequential(
                nn.Linear(input_dim, 256),
                nn.ReLU(),
                nn.Dropout(0.5),
                nn.Linear(256, 128),
                nn.ReLU(),
                nn.Dropout(0.5),
                nn.Linear(128, 2) # Output 2 logits for binary classification
            )

        def forward(self, x):
            return self.model(x)

# --- Load pre-trained models for face detection and embedding ---
mtcnn = MTCNN(image_size=160, margin=20, keep_all=False, device=device, post_process=True)
facenet: InceptionResnetV1 = InceptionResnetV1(pretrained='vggface2', device=device)

def get_embedding(image_path: str) -> Tensor | None:
    '''Extracts a face embedding from a given image path.'''
    try:
        img = Image.open(DATA_DIR + image_path).convert('RGB')
    except FileNotFoundError:
        print(f"Error: Could not find image {image_path}")
        return None

    return get_embeddings_from_image(img)

```

```

def get_embeddings_from_image(image: Image.Image) -> Tensor | None:
    '''Extracts a face embedding from a given PIL Image.'''
    face_tensor = mtcnn(image)
    if face_tensor is None:
        return None # Skip images where no face is detected

    face_tensor = face_tensor.to(device)
    embedding = facenet(face_tensor.unsqueeze(0))
    return embedding.detach()

def get_diff_vector(img1_path: str, img2_path: str) -> Tensor | None:
    '''Calculates the absolute difference vector between two image embeddings.'''
    emb1 = get_embedding(img1_path)
    emb2 = get_embedding(img2_path)
    return get_diff_vector_from_embeddings(emb1, emb2)

def get_diff_vector_from_embeddings(image_1_embeddings: Tensor | None, image_2_embeddings: Tensor | None):
    '''Calculates the absolute difference vector between two image embeddings.'''
    if image_1_embeddings is None or image_2_embeddings is None:
        return None
    return torch.abs(image_1_embeddings - image_2_embeddings)

def augment_image(image, augment_type="gaussian_noise"):
    '''Example augmentation function'''
    if augment_type == "gaussian_noise":
        image_np = np.array(image).astype(np.float32)
        noise = np.random.normal(0, 25, image_np.shape)
        noisy_image = image_np + noise
        noisy_image = np.clip(noisy_image, 0, 255).astype(np.uint8)
        augmented = Image.fromarray(noisy_image)
    elif augment_type == "blur":
        augmented = image.filter(ImageFilter.GaussianBlur(radius=3))
    elif augment_type == "increased_lighting":
        enhancer = ImageEnhance.Brightness(image)
        augmented = enhancer.enhance(1.5)
    else:
        augmented = image
    return augmented

```

In [7]: `from torch._tensor import Tensor`

```

def generate_embeddings_for_pairs(pairs: List[Tuple[str, str, int]]):
    X_train, y_train = [], []
    print("Generating embeddings for training data...")
    for i, (img1, img2, label) in enumerate(pairs):
        print(f" Processing pair {i+1}/{len(pairs)}...", end='\r')
        diff = get_diff_vector(img1, img2)
        if diff is not None:
            X_train.append(diff.squeeze(0))
            y_train.append(label)

    return X_train, y_train

def train_model(
    X_train,
    y_train,
    lr: float,
    epochs: int,
    batch_size: int = 64

```

```

) -> FaceVerificationMLP:
    """
    Trains an MLP model from scratch given training data and hyperparameters.
    """

    if not X_train:
        raise ValueError("Could not generate any valid training embeddings.")

    X_train_tensor = torch.stack(X_train).to(device)
    y_train_tensor = torch.tensor(y_train, dtype=torch.long).to(device)

    model = FaceVerificationMLP().to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    print(f"Starting model training: {epochs} epochs, lr={lr}, batch_size={batch_size}")
    for epoch in range(epochs):
        model.train()
        permutation = torch.randperm(X_train_tensor.size()[0])

        for i in range(0, X_train_tensor.size()[0], batch_size):
            optimizer.zero_grad()
            indices = permutation[i:i+batch_size]
            batch_X, batch_y = X_train_tensor[indices], y_train_tensor[indices]

            outputs = model(batch_X)
            loss = criterion(outputs, batch_y)
            loss.backward()
            optimizer.step()

            if (epoch + 1) % 5 == 0 or (epoch + 1) == epochs:
                print(f" Epoch {epoch+1}/{epochs} completed.")

    return model

def get_diffs(
    pairs: List[Tuple[str, str, int]],
    augment_type: str|None=None
) -> List[Tuple[Tensor, int]]:
    """
    Generates difference vectors for a list of image pairs.
    Optionally applies augmentation to the images.
    """

    diffs = []
    print("Generating difference vectors...")

    for i, (img1, img2, label) in enumerate(pairs):
        print(f" Processing pair {i+1}/{len(pairs)}...", end='\r')
        if augment_type is not None:
            img1 = augment_image(Image.open(DATA_DIR + img1), augment_type)
            img2 = augment_image(Image.open(DATA_DIR + img2), augment_type)
            diff = get_diff_vector_from_embeddings(
                get_embeddings_from_image(img1), get_embeddings_from_image(img2)
            )
        else:
            diff = get_diff_vector(img1, img2)

        if diff is not None:
            diffs.append((diff, label))

```

```

    return diffs

def evaluate_model(model: nn.Module, diffs: List[Tuple[Tensor, int]], augment_ty
    """
    Evaluates a trained model on a validation set and returns performance metrics
    """
    print("Evaluating model...")
    tp, fp, tn, fn = 0, 0, 0, 0
    model.eval()

    with torch.no_grad():
        for i, (diff, true_label) in enumerate(diffs):
            print(f" Evaluating pair {i+1}/{len(diffs)}", end='\r')

            diff = diff.to(device)
            output = model(diff)
            _, predicted_label = torch.max(output, 1)
            predicted_label = predicted_label.item()

            if predicted_label == 1 and true_label == 1: tp += 1
            elif predicted_label == 1 and true_label == 0: fp += 1
            elif predicted_label == 0 and true_label == 0: tn += 1
            elif predicted_label == 0 and true_label == 1: fn += 1

    print("\nEvaluation complete.")

    total = tp + fp + tn + fn
    epsilon = 1e-6
    accuracy = (tp + tn) / (total + epsilon)
    precision = tp / (tp + fp + epsilon)
    recall = tp / (tp + fn + epsilon)
    f1_score = 2 * (precision * recall) / (precision + recall + epsilon)

    return {
        'accuracy': accuracy, 'precision': precision, 'recall': recall, 'f1_score': f1_score,
        'tp': tp, 'fp': fp, 'tn': tn, 'fn': fn
    }

```

Zadanie 1: Analiza wielkości training set size na wyniki modelu

```

In [8]: train_sizes = [10, 100, 300, 500, 700, 1000, 5000]
        VALIDATION_SET_SIZE = 200
        FIXED_LR = 0.001
        FIXED_EPOCHS = 20
        results_task1 = {}

        for n in train_sizes:
            print("-" * 60)
            print(f"STARTING EXPERIMENT WITH N = {n} TRAINING PAIRS")

            print("\nStep 1: Generating data...")
            train_data = get_balanced_training_data(n)
            exclude_images = {img for pair in train_data for img in pair[:2]}
            validation_data = get_balanced_training_data(VALIDATION_SET_SIZE, exclude_images)
            print(f"Generated {len(train_data)} training pairs and {len(validation_data)} validation pairs")

            X_train, y_train = generate_embeddings_for_pairs(train_data)

            validation_diffs = get_diffs(validation_data)

```

```

print("\nStep 2: Training model...")
model = train_model(X_train, y_train, lr=FIXED_LR, epochs=FIXED_EPOCHS)

print("\nStep 3: Evaluating model...")
metrics = evaluate_model(model, validation_diffs)
results_task1[n] = metrics

print("\n--- Metrics for N =", n, "---")
print(f" Accuracy: {metrics['accuracy']:.2%}")
print(f" Precision: {metrics['precision']:.2%}")
print(f" Recall: {metrics['recall']:.2%}")
print(f" F1-Score: {metrics['f1_score']:.2%}")
print(f" (TP: {metrics['tp']}, FP: {metrics['fp']}, TN: {metrics['tn']}, FN: {metrics['fn']})")

print("\n" + "="*70)
print(" " * 15 + "TASK 1: FINAL RESULTS SUMMARY (TRAINING SET SIZE)" + " " * 15)
print("="*70)
print(f"{'N (Train Pairs)':>16} | {'Accuracy':>12} | {'Precision':>12} | {'Recall':>12} | {'F1-Score':>12} | {'TP':>12} | {'FP':>12} | {'TN':>12} | {'FN':>12}")
print("-" * 70)
for n, metrics in results_task1.items():
    print(f"{n:>16} | {metrics['accuracy']:>12.2%} | {metrics['precision']:>12.2%} | {metrics['recall']:>12.2%} | {metrics['f1_score']:>12.2%} | {metrics['tp']:>12} | {metrics['fp']:>12} | {metrics['tn']:>12} | {metrics['fn']:>12}")
print("="*70)

```

STARTING EXPERIMENT WITH N = 10 TRAINING PAIRS

Step 1: Generating data...

Generated 10 training pairs and 200 validation pairs.

Generating embeddings for training data...

Generating difference vectors...

Processing pair 200/200...

Step 2: Training model...

Starting model training: 20 epochs, lr=0.001, batch_size=64

Epoch 5/20 completed.

Epoch 10/20 completed.

Epoch 15/20 completed.

Epoch 20/20 completed.

Step 3: Evaluating model...

Evaluating model...

Evaluating pair 199/199

Evaluation complete.

--- Metrics for N = 10 ---

Accuracy: 54.77%

Precision: 53.47%

Recall: 77.00%

F1-Score: 63.11%

(TP: 77, FP: 67, TN: 32, FN: 23)

STARTING EXPERIMENT WITH N = 100 TRAINING PAIRS

Step 1: Generating data...

Generated 100 training pairs and 200 validation pairs.

Generating embeddings for training data...

Generating difference vectors...

Processing pair 200/200...

Step 2: Training model...

Starting model training: 20 epochs, lr=0.001, batch_size=64

Epoch 5/20 completed.

Epoch 10/20 completed.

Epoch 15/20 completed.

Epoch 20/20 completed.

Step 3: Evaluating model...

Evaluating model...

Evaluating pair 198/198

Evaluation complete.

--- Metrics for N = 100 ---

Accuracy: 55.05%

Precision: 56.58%

Recall: 43.43%

F1-Score: 49.14%

(TP: 43, FP: 33, TN: 66, FN: 56)

STARTING EXPERIMENT WITH N = 300 TRAINING PAIRS

Step 1: Generating data...

Generated 300 training pairs and 200 validation pairs.

Generating embeddings for training data...

Generating difference vectors...

Processing pair 200/200...

Step 2: Training model...
Starting model training: 20 epochs, lr=0.001, batch_size=64
Epoch 5/20 completed.
Epoch 10/20 completed.
Epoch 15/20 completed.
Epoch 20/20 completed.

Step 3: Evaluating model...
Evaluating model...
Evaluating pair 198/198
Evaluation complete.

--- Metrics for N = 300 ---
Accuracy: 53.54%
Precision: 52.67%
Recall: 69.70%
F1-Score: 60.00%
(TP: 69, FP: 62, TN: 37, FN: 30)

STARTING EXPERIMENT WITH N = 500 TRAINING PAIRS

Step 1: Generating data...
Generated 500 training pairs and 200 validation pairs.
Generating embeddings for training data...
Generating difference vectors...
Processing pair 200/200...
Step 2: Training model...
Starting model training: 20 epochs, lr=0.001, batch_size=64
Epoch 5/20 completed.
Epoch 10/20 completed.
Epoch 15/20 completed.
Epoch 20/20 completed.

Step 3: Evaluating model...
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for N = 500 ---
Accuracy: 61.31%
Precision: 60.58%
Recall: 63.64%
F1-Score: 62.07%
(TP: 63, FP: 41, TN: 59, FN: 36)

STARTING EXPERIMENT WITH N = 700 TRAINING PAIRS

Step 1: Generating data...
Generated 700 training pairs and 200 validation pairs.
Generating embeddings for training data...
Generating difference vectors...
Processing pair 200/200...
Step 2: Training model...
Starting model training: 20 epochs, lr=0.001, batch_size=64
Epoch 5/20 completed.
Epoch 10/20 completed.
Epoch 15/20 completed.
Epoch 20/20 completed.

Step 3: Evaluating model...

Evaluating model...

Evaluating pair 200/200

Evaluation complete.

--- Metrics for N = 700 ---

Accuracy: 62.00%

Precision: 61.32%

Recall: 65.00%

F1-Score: 63.11%

(TP: 65, FP: 41, TN: 59, FN: 35)

STARTING EXPERIMENT WITH N = 1000 TRAINING PAIRS

Step 1: Generating data...

Generated 1000 training pairs and 200 validation pairs.

Generating embeddings for training data...

Generating difference vectors...

Processing pair 200/200...

Step 2: Training model...

Starting model training: 20 epochs, lr=0.001, batch_size=64

Epoch 5/20 completed.

Epoch 10/20 completed.

Epoch 15/20 completed.

Epoch 20/20 completed.

Step 3: Evaluating model...

Evaluating model...

Evaluating pair 200/200

Evaluation complete.

--- Metrics for N = 1000 ---

Accuracy: 56.50%

Precision: 56.19%

Recall: 59.00%

F1-Score: 57.56%

(TP: 59, FP: 46, TN: 54, FN: 41)

STARTING EXPERIMENT WITH N = 5000 TRAINING PAIRS

Step 1: Generating data...

Generated 5000 training pairs and 200 validation pairs.

Generating embeddings for training data...

Generating difference vectors...

Processing pair 200/200...

Step 2: Training model...

Starting model training: 20 epochs, lr=0.001, batch_size=64

Epoch 5/20 completed.

Epoch 10/20 completed.

Epoch 15/20 completed.

Epoch 20/20 completed.

Step 3: Evaluating model...

Evaluating model...

Evaluating pair 199/199

Evaluation complete.

--- Metrics for N = 5000 ---

Accuracy: 63.82%

Precision: 65.17%

Recall: 58.59%

F1-Score: 61.70%
(TP: 58, FP: 31, TN: 69, FN: 41)

TASK 1: FINAL RESULTS SUMMARY (TRAINING SET SIZE)					
N (Train Pairs)	Accuracy	Precision	Recall	F1-Score	
10	54.77%	53.47%	77.00%	63.11%	
100	55.05%	56.58%	43.43%	49.14%	
300	53.54%	52.67%	69.70%	60.00%	
500	61.31%	60.58%	63.64%	62.07%	
700	62.00%	61.32%	65.00%	63.11%	
1000	56.50%	56.19%	59.00%	57.56%	
5000	63.82%	65.17%	58.59%	61.70%	

Co ciekawe, najwyższy F1-Score ma model wytrenowany na N=700. Wysoki wynik osiągnął również N=500 i N=5000.

Najwyższe accuracy było dla N=5000, potem N=700 i N=500.

```
In [9]: fixed_train_data_lr: List[Tuple[str, str, int]] = get_balanced_training_data(1000,
exclude_images_lr: set[str] = {img for pair in fixed_train_data_lr for img in pair}
fixed_validation_data_lr = get_balanced_training_data(VALIDATION_SET_SIZE, exclude_images_lr)
print(f"Generated fixed dataset: {len(fixed_train_data_lr)} training pairs, {len(fixed_validation_data_lr)} validation pairs")

X_train, y_train = generate_embeddings_for_pairs(fixed_train_data_lr)

validation_diffs = get_diffs(fixed_validation_data_lr)
```

Generated fixed dataset: 1000 training pairs, 200 validation pairs.

Generating embeddings for training data...

Generating difference vectors...

Processing pair 200/200...

Zadanie 2: testy learning rate

```
In [10]: FIXED_EPOCHS_FOR_LR = 20
learning_rates = [1e-6, 1e-5, 1e-4, 5e-4, 8e-4, 1e-3, 2e-3, 3e-3, 4e-3, 5e-3, 1e-2]
results_task2 = {}

for lr in learning_rates:
    print("-" * 60)
    print(f"STARTING EXPERIMENT WITH LEARNING RATE = {lr}")

    model = train_model(X_train, y_train, lr=lr, epochs=FIXED_EPOCHS_FOR_LR)

    metrics = evaluate_model(model, validation_diffs)
    results_task2[lr] = metrics

    print("\n--- Metrics for LR =", lr, "---")
    print(f"Accuracy: {metrics['accuracy']:.2%}")
    print(f"F1-Score: {metrics['f1_score']:.2%}")

print("\n" + "="*70)
print(" " * 17 + "TASK 2: FINAL RESULTS SUMMARY (LEARNING RATE)" + " " * 18)
print("="*70)
```

```
print(f"{'Learning Rate':>16} | {'Accuracy':>12} | {'Precision':>12} | {'Recall':>12}")
print("-" * 70)
for lr, metrics in results_task2.items():
    print(f"{lr:>16.5f} | {metrics['accuracy']:>12.2%} | {metrics['precision']:>12.2%} | {metrics['recall']:>12.2%}")
print("=" * 70)
```

```
-----
STARTING EXPERIMENT WITH LEARNING RATE = 1e-06
Starting model training: 20 epochs, lr=1e-06, batch_size=64
  Epoch 5/20 completed.
  Epoch 10/20 completed.
  Epoch 15/20 completed.
  Epoch 20/20 completed.
Evaluating model...
  Evaluating pair 199/199
Evaluation complete.

--- Metrics for LR = 1e-06 ---
  Accuracy:  49.75%
  F1-Score:  66.44%
-----

STARTING EXPERIMENT WITH LEARNING RATE = 1e-05
Starting model training: 20 epochs, lr=1e-05, batch_size=64
  Epoch 5/20 completed.
  Epoch 10/20 completed.
  Epoch 15/20 completed.
  Epoch 20/20 completed.
Evaluating model...
  Evaluating pair 199/199
Evaluation complete.

--- Metrics for LR = 1e-05 ---
  Accuracy:  50.25%
  F1-Score:  0.00%
-----

STARTING EXPERIMENT WITH LEARNING RATE = 0.0001
Starting model training: 20 epochs, lr=0.0001, batch_size=64
  Epoch 5/20 completed.
  Epoch 10/20 completed.
  Epoch 15/20 completed.
  Epoch 20/20 completed.
Evaluating model...
  Evaluating pair 199/199
Evaluation complete.

--- Metrics for LR = 0.0001 ---
  Accuracy:  57.29%
  F1-Score:  37.96%
-----

STARTING EXPERIMENT WITH LEARNING RATE = 0.0005
Starting model training: 20 epochs, lr=0.0005, batch_size=64
  Epoch 5/20 completed.
  Epoch 10/20 completed.
  Epoch 15/20 completed.
  Epoch 20/20 completed.
Evaluating model...
  Evaluating pair 199/199
Evaluation complete.

--- Metrics for LR = 0.0005 ---
  Accuracy:  56.28%
  F1-Score:  62.34%
-----

STARTING EXPERIMENT WITH LEARNING RATE = 0.0008
Starting model training: 20 epochs, lr=0.0008, batch_size=64
  Epoch 5/20 completed.
```

Epoch 10/20 completed.
Epoch 15/20 completed.
Epoch 20/20 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for LR = 0.0008 ---
Accuracy: 57.29%
F1-Score: 62.88%

STARTING EXPERIMENT WITH LEARNING RATE = 0.001
Starting model training: 20 epochs, lr=0.001, batch_size=64
Epoch 5/20 completed.
Epoch 10/20 completed.
Epoch 15/20 completed.
Epoch 20/20 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for LR = 0.001 ---
Accuracy: 60.30%
F1-Score: 61.08%

STARTING EXPERIMENT WITH LEARNING RATE = 0.002
Starting model training: 20 epochs, lr=0.002, batch_size=64
Epoch 5/20 completed.
Epoch 10/20 completed.
Epoch 15/20 completed.
Epoch 20/20 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for LR = 0.002 ---
Accuracy: 60.80%
F1-Score: 55.17%

STARTING EXPERIMENT WITH LEARNING RATE = 0.003
Starting model training: 20 epochs, lr=0.003, batch_size=64
Epoch 5/20 completed.
Epoch 10/20 completed.
Epoch 15/20 completed.
Epoch 20/20 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for LR = 0.003 ---
Accuracy: 54.27%
F1-Score: 64.31%

STARTING EXPERIMENT WITH LEARNING RATE = 0.004
Starting model training: 20 epochs, lr=0.004, batch_size=64
Epoch 5/20 completed.
Epoch 10/20 completed.
Epoch 15/20 completed.
Epoch 20/20 completed.
Evaluating model...

Evaluating pair 199/199
Evaluation complete.

--- Metrics for LR = 0.004 ---
Accuracy: 58.79%
F1-Score: 58.59%

STARTING EXPERIMENT WITH LEARNING RATE = 0.005
Starting model training: 20 epochs, lr=0.005, batch_size=64
Epoch 5/20 completed.
Epoch 10/20 completed.
Epoch 15/20 completed.
Epoch 20/20 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for LR = 0.005 ---
Accuracy: 62.81%
F1-Score: 56.98%

STARTING EXPERIMENT WITH LEARNING RATE = 0.01
Starting model training: 20 epochs, lr=0.01, batch_size=64
Epoch 5/20 completed.
Epoch 10/20 completed.
Epoch 15/20 completed.
Epoch 20/20 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for LR = 0.01 ---
Accuracy: 60.80%
F1-Score: 50.63%

STARTING EXPERIMENT WITH LEARNING RATE = 0.1
Starting model training: 20 epochs, lr=0.1, batch_size=64
Epoch 5/20 completed.
Epoch 10/20 completed.
Epoch 15/20 completed.
Epoch 20/20 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for LR = 0.1 ---
Accuracy: 50.25%
F1-Score: 0.00%

STARTING EXPERIMENT WITH LEARNING RATE = 1
Starting model training: 20 epochs, lr=1, batch_size=64
Epoch 5/20 completed.
Epoch 10/20 completed.
Epoch 15/20 completed.
Epoch 20/20 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for LR = 1 ---

Accuracy: 50.25%
F1-Score: 0.00%

TASK 2: FINAL RESULTS SUMMARY (LEARNING RATE)				
Learning Rate	Accuracy	Precision	Recall	F1-Score
0.00000	49.75%	49.75%	100.00%	66.44%
0.00001	50.25%	0.00%	0.00%	0.00%
0.00010	57.29%	68.42%	26.26%	37.96%
0.00050	56.28%	54.55%	72.73%	62.34%
0.00080	57.29%	55.38%	72.73%	62.88%
0.00100	60.30%	59.62%	62.63%	61.08%
0.00200	60.80%	64.00%	48.48%	55.17%
0.00300	54.27%	52.56%	82.83%	64.31%
0.00400	58.79%	58.59%	58.59%	58.59%
0.00500	62.81%	67.12%	49.49%	56.98%
0.01000	60.80%	67.80%	40.40%	50.63%
0.10000	50.25%	0.00%	0.00%	0.00%
1.00000	50.25%	0.00%	0.00%	0.00%

Jeśli celem jest najlepszy ogólny balans, to najlepszy jest $lr = 0.00300$, ponieważ osiąga najwyższy F1-Score (64.31%). Trzeba jednak pamiętać o jego skłonności do wysokiej czułości i niższej precyzji. Bardzo dobrym, bezpieczniejszym kandydatem jest $lr = 0.00080$. Ma on drugi najwyższy F1-Score (62.88%), ale jego precyzja (55.38%) i czułość (72.73%) są znacznie lepiej zrównoważone niż w przypadku $lr = 0.00300$.

Model, co najrzadziej będzie popełniał błąd "false positive" jest model o $lr = 0.00010$ z najwyższą precyzją (68.42%).

Model, który wykryje najwięcej prawdziwych par jest model o $lr = 0.00300$ z najwyższą czułością (82.83%).

Zadanie 3: epochs

```
In [14]: FIXED_LR_FOR_EPOCH = 1e-3
EPOCH_COUNT = [5, 7, 9, 11, 13, 15, 17, 20, 25, 30, 50]
results_task3 = {}

for epochs in EPOCH_COUNT:
    print("-" * 60)
    print(f"STARTING EXPERIMENT WITH EPOCH COUNT = {epochs}")

    model = train_model(X_train, y_train, lr=FIXED_LR_FOR_EPOCH, epochs=epochs)

    metrics = evaluate_model(model, validation_diffs)
    results_task3[epochs] = metrics

    print("\n--- Metrics for Epochs =", epochs, "---")
    print(f"  Accuracy: {metrics['accuracy']:.2%}")
    print(f"  F1-Score: {metrics['f1_score']:.2%}")

print("\n" + "="*70)
print(" " * 17 + "TASK 3: FINAL RESULTS SUMMARY (NUMBER OF EPOCHS)" + " " * 16)
print("="*70)
```

```
print(f"{'Num. of Epochs':>16} | {'Accuracy':>12} | {'Precision':>12} | {'Recall':>12}")
print("-" * 70)
for epochs, metrics in results_task3.items():
    print(f"{'epochs':>16} | {'metrics['accuracy']':>12.2%} | {'metrics['precision']':>12.2%} | {'metrics['recall']':>12.2%}")
print("=" * 70)
```

STARTING EXPERIMENT WITH EPOCH COUNT = 5
Starting model training: 5 epochs, lr=0.001, batch_size=64
Epoch 5/5 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for Epochs = 5 ---
Accuracy: 58.29%
F1-Score: 54.14%

STARTING EXPERIMENT WITH EPOCH COUNT = 7
Starting model training: 7 epochs, lr=0.001, batch_size=64
Epoch 5/7 completed.
Epoch 7/7 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for Epochs = 7 ---
Accuracy: 56.78%
F1-Score: 60.91%

STARTING EXPERIMENT WITH EPOCH COUNT = 9
Starting model training: 9 epochs, lr=0.001, batch_size=64
Epoch 5/9 completed.
Epoch 9/9 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for Epochs = 9 ---
Accuracy: 55.28%
F1-Score: 58.22%

STARTING EXPERIMENT WITH EPOCH COUNT = 11
Starting model training: 11 epochs, lr=0.001, batch_size=64
Epoch 5/11 completed.
Epoch 10/11 completed.
Epoch 11/11 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for Epochs = 11 ---
Accuracy: 60.30%
F1-Score: 50.31%

STARTING EXPERIMENT WITH EPOCH COUNT = 13
Starting model training: 13 epochs, lr=0.001, batch_size=64
Epoch 5/13 completed.
Epoch 10/13 completed.
Epoch 13/13 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for Epochs = 13 ---
Accuracy: 56.28%

F1-Score: 58.37%

STARTING EXPERIMENT WITH EPOCH COUNT = 15
Starting model training: 15 epochs, lr=0.001, batch_size=64
Epoch 5/15 completed.
Epoch 10/15 completed.
Epoch 15/15 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for Epochs = 15 ---
Accuracy: 60.80%
F1-Score: 55.17%

STARTING EXPERIMENT WITH EPOCH COUNT = 17
Starting model training: 17 epochs, lr=0.001, batch_size=64
Epoch 5/17 completed.
Epoch 10/17 completed.
Epoch 15/17 completed.
Epoch 17/17 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for Epochs = 17 ---
Accuracy: 58.79%
F1-Score: 60.95%

STARTING EXPERIMENT WITH EPOCH COUNT = 20
Starting model training: 20 epochs, lr=0.001, batch_size=64
Epoch 5/20 completed.
Epoch 10/20 completed.
Epoch 15/20 completed.
Epoch 20/20 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for Epochs = 20 ---
Accuracy: 57.79%
F1-Score: 60.75%

STARTING EXPERIMENT WITH EPOCH COUNT = 25
Starting model training: 25 epochs, lr=0.001, batch_size=64
Epoch 5/25 completed.
Epoch 10/25 completed.
Epoch 15/25 completed.
Epoch 20/25 completed.
Epoch 25/25 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

--- Metrics for Epochs = 25 ---
Accuracy: 61.31%
F1-Score: 62.07%

STARTING EXPERIMENT WITH EPOCH COUNT = 30
Starting model training: 30 epochs, lr=0.001, batch_size=64

```

Epoch 5/30 completed.
Epoch 10/30 completed.
Epoch 15/30 completed.
Epoch 20/30 completed.
Epoch 25/30 completed.
Epoch 30/30 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

```

```

--- Metrics for Epochs = 30 ---
Accuracy: 59.30%
F1-Score: 56.68%

```

```

-----
STARTING EXPERIMENT WITH EPOCH COUNT = 50
Starting model training: 50 epochs, lr=0.001, batch_size=64
Epoch 5/50 completed.
Epoch 10/50 completed.
Epoch 15/50 completed.
Epoch 20/50 completed.
Epoch 25/50 completed.
Epoch 30/50 completed.
Epoch 35/50 completed.
Epoch 40/50 completed.
Epoch 45/50 completed.
Epoch 50/50 completed.
Evaluating model...
Evaluating pair 199/199
Evaluation complete.

```

```

--- Metrics for Epochs = 50 ---
Accuracy: 55.28%
F1-Score: 50.83%

```

TASK 3: FINAL RESULTS SUMMARY (NUMBER OF EPOCHS)				
Num. of Epochs	Accuracy	Precision	Recall	F1-Score
5	58.29%	59.76%	49.49%	54.14%
7	56.78%	55.37%	67.68%	60.91%
9	55.28%	54.39%	62.63%	58.22%
11	60.30%	66.67%	40.40%	50.31%
13	56.28%	55.45%	61.62%	58.37%
15	60.80%	64.00%	48.48%	55.17%
17	58.79%	57.66%	64.65%	60.95%
20	57.79%	56.52%	65.66%	60.75%
25	61.31%	60.58%	63.64%	62.07%
30	59.30%	60.23%	53.54%	56.68%
50	55.28%	56.10%	46.46%	50.83%

Najlepsze F1-Score osiągnięty był przy liczbie epochs równej 25 - osiągnął wysokie wartości we wszystkich metrykach

Zadanie 4: najlepsze hiperparametry

```

In [31]: FIXED_LR_FOR_EPOCH = 3e-3
          EPOCH_COUNT = 25

```

```

model = train_model(X_train, y_train, lr=FIXED_LR_FOR_EPOCH, epochs=EPOCH_COUNT)

metrics = evaluate_model(model, validation_diffs)

print("\n" + "="*70)
print(" " * 17 + "TASK 4: FINAL RESULTS SUMMARY" + " " * 16)
print("="*70)
print(f"{metrics['accuracy']:>12.2%} | {metrics['precision']:>12.2%} | {metrics[

```

Starting model training: 25 epochs, lr=0.003, batch_size=64

Epoch 5/25 completed.

Epoch 10/25 completed.

Epoch 15/25 completed.

Epoch 20/25 completed.

Epoch 25/25 completed.

Evaluating model...

Evaluating pair 199/199

Evaluation complete.

```

=====
                        TASK 4: FINAL RESULTS SUMMARY
=====
55.78% | 53.74% | 79.80% | 64.23%

```

Zadanie 5: Analiza odporności na zakłócenia i strategia mitygacji

```

In [32]: OPTIMAL_N = 700
          OPTIMAL_LR = 0.003
          OPTIMAL_EPOCHS = 25
          VALIDATION_SET_SIZE = 200

          print("Generating fixed datasets for experiments...")
          fixed_train_data = get_balanced_training_data(OPTIMAL_N)
          exclude_images = {img for pair in fixed_train_data for img in pair[:2]}
          fixed_validation_data = get_balanced_training_data(VALIDATION_SET_SIZE, exclude_
          print(f"Generated fixed dataset: {len(fixed_train_data)} training pairs, {len(fi

```

Generating fixed datasets for experiments...

Generated fixed dataset: 700 training pairs, 200 validation pairs.

Model trenowany na czystych danych

```

In [ ]: print("-" * 60)
          print("PART 1: TRAINING BASELINE MODEL ON CLEAN DATA")
          print("-" * 60)

          X_train_vanilla, y_train_vanilla = generate_embeddings_for_pairs(fixed_train_dat
          vanilla_model = train_model(X_train_vanilla, y_train_vanilla, lr=OPTIMAL_LR, epo

```

PART 1: TRAINING BASELINE MODEL ON CLEAN DATA

Generating embeddings for training data...

Starting model training: 25 epochs, lr=0.003, batch_size=64

Epoch 5/25 completed.

Epoch 10/25 completed.

Epoch 15/25 completed.

Epoch 20/25 completed.

Epoch 25/25 completed.

```
In [ ]: print("\nGenerating difference vectors for all validation sets...")
validation_diffs_original = get_diffs(fixed_validation_data, augment_type=None)
validation_diffs_noise = get_diffs(fixed_validation_data, augment_type="gaussian")
validation_diffs_blur = get_diffs(fixed_validation_data, augment_type="blur")
validation_diffs_light = get_diffs(fixed_validation_data, augment_type="increase")

results_vanilla = {}
print("\n" + "="*70)
print("EVALUATING BASELINE MODEL (TRAINED ON CLEAN DATA)")
print("="*70)

print("\nEvaluating on ORIGINAL validation data...")
results_vanilla['Original'] = evaluate_model(vanilla_model, validation_diffs_original)

print("\nEvaluating on GAUSSIAN NOISE validation data...")
results_vanilla['Gaussian Noise'] = evaluate_model(vanilla_model, validation_diffs_noise)

print("\nEvaluating on BLUR validation data...")
results_vanilla['Blur'] = evaluate_model(vanilla_model, validation_diffs_blur)

print("\nEvaluating on INCREASED LIGHTING validation data...")
results_vanilla['Increased Lighting'] = evaluate_model(vanilla_model, validation_diffs_light)

print("\n" + "="*70)
print(" " * 10 + "BASELINE MODEL PERFORMANCE SUMMARY" + " " * 10)
print("="*70)
print(f"{'Validation Set':<20} | {'Accuracy':>12} | {'Precision':>12} | {'Recall':>12}")
print("-" * 70)
for name, metrics in results_vanilla.items():
    print(f"{'name':<20} | {metrics['accuracy']:>12.2%} | {metrics['precision']:>12.2%} | {metrics['recall']:>12.2%}")
print("="*70)
```

```

Generating difference vectors for all validation sets...
Generating difference vectors...
Generating difference vectors...
Generating difference vectors...
Generating difference vectors...
Processing pair 200/200...

```

```

=====
EVALUATING BASELINE MODEL (TRAINED ON CLEAN DATA)
=====

```

```

Evaluating on ORIGINAL validation data...
Evaluating model...
    Evaluating pair 199/199
Evaluation complete.

```

```

Evaluating on GAUSSIAN NOISE validation data...
Evaluating model...
    Evaluating pair 199/199
Evaluation complete.

```

```

Evaluating on BLUR validation data...
Evaluating model...
    Evaluating pair 198/198
Evaluation complete.

```

```

Evaluating on INCREASED LIGHTING validation data...
Evaluating model...
    Evaluating pair 187/187
Evaluation complete.

```

```

=====
BASELINE MODEL PERFORMANCE SUMMARY
=====

```

Validation Set	Accuracy	Precision	Recall	F1-Score
Original	56.78%	56.07%	60.61%	58.25%
Gaussian Noise	55.78%	55.24%	58.59%	56.86%
Blur	54.04%	59.46%	22.45%	32.59%
Increased Lighting	57.22%	57.30%	54.84%	56.04%

Model "vanilla" poradził sobie słabo przez augmentację, zwłaszcza w kategorii blur - ogromny spadek recall.

```

In [ ]: def generate_augmented_embeddings(
        pairs: List[Tuple[str, str, int]],
        augment_types: List[str]
    ) -> Tuple[List[Tensor], List[int]]:
        """
        Generates difference vectors for training, applying a random augmentation
        from the provided list to each image in a pair.
        """
        X_train, y_train = [], []
        print("Generating augmented embeddings for training data...")

        for i, (img1_path, img2_path, label) in enumerate(pairs):
            print(f" Processing pair {i+1}/{len(pairs)}...", end='\r')

            try:

```

```

img1_pil = Image.open(DATA_DIR + img1_path).convert('RGB')
img2_pil = Image.open(DATA_DIR + img2_path).convert('RGB')

aug_type1 = random.choice(augment_types)
aug_type2 = random.choice(augment_types)

aug_img1 = augment_image(img1_pil, aug_type1)
aug_img2 = augment_image(img2_pil, aug_type2)

diff = get_diff_vector_from_embeddings(
    get_embeddings_from_image(aug_img1),
    get_embeddings_from_image(aug_img2)
)

if diff is not None:
    X_train.append(diff.squeeze(0))
    y_train.append(label)
except FileNotFoundError:
    continue

print("\nGeneration of augmented embeddings complete.")
return X_train, y_train

print("\n" + "-" * 60)
print("PART 2: TRAINING AUGMENTED MODEL")
print("-" * 60)

augment_list = ["gaussian_noise", "blur", "increased_lighting", "none"]

X_train_aug, y_train_aug = generate_augmented_embeddings(fixed_train_data, augme
augmented_model = train_model(X_train_aug, y_train_aug, lr=OPTIMAL_LR, epochs=OP

```

PART 2: TRAINING AUGMENTED MODEL

Generating augmented embeddings for training data...

Processing pair 700/700...

Generation of augmented embeddings complete.

Starting model training: 25 epochs, lr=0.003, batch_size=64

Epoch 5/25 completed.

Epoch 10/25 completed.

Epoch 15/25 completed.

Epoch 20/25 completed.

Epoch 25/25 completed.

```

In [ ]: results_augmented = {}
print("\n" + "="*70)
print("EVALUATING AUGMENTED MODEL (TRAINED ON AUGMENTED DATA)")
print("="*70)

print("\nEvaluating on ORIGINAL validation data...")
results_augmented['Original'] = evaluate_model(augmented_model, validation_diffs

print("\nEvaluating on GAUSSIAN NOISE validation data...")
results_augmented['Gaussian Noise'] = evaluate_model(augmented_model, validation

print("\nEvaluating on BLUR validation data...")
results_augmented['Blur'] = evaluate_model(augmented_model, validation_diffs_blu

```

```

print("\nEvaluating on INCREASED LIGHTING validation data...")
results_augmented['Increased Lighting'] = evaluate_model(augmented_model, valida

print("\n" + "="*70)
print(" " * 10 + "AUGMENTED MODEL PERFORMANCE SUMMARY" + " " * 10)
print("="*70)
print(f"{'Validation Set':<20} | {'Accuracy':>12} | {'Precision':>12} | {'Recall'
print("-" * 70)
for name, metrics in results_augmented.items():
    print(f"{'name':<20} | {metrics['accuracy']:>12.2%} | {metrics['precision']:>1
print("="*70)

```

```

=====
EVALUATING AUGMENTED MODEL (TRAINED ON AUGMENTED DATA)
=====

```

Evaluating on ORIGINAL validation data...

Evaluating model...

Evaluating pair 199/199

Evaluation complete.

Evaluating on GAUSSIAN NOISE validation data...

Evaluating model...

Evaluating pair 199/199

Evaluation complete.

Evaluating on BLUR validation data...

Evaluating model...

Evaluating pair 198/198

Evaluation complete.

Evaluating on INCREASED LIGHTING validation data...

Evaluating model...

Evaluating pair 187/187

Evaluation complete.

```

=====
AUGMENTED MODEL PERFORMANCE SUMMARY
=====

```

Validation Set	Accuracy	Precision	Recall	F1-Score
Original	54.27%	55.13%	43.43%	48.59%
Gaussian Noise	58.79%	61.04%	47.47%	53.41%
Blur	49.49%	48.15%	26.53%	34.21%
Increased Lighting	58.82%	61.43%	46.24%	52.76%

```

=====

```

```

In [41]: print("\n" + "="*80)
print(" " * 18 + "FINAL COMPARISON: VANILLA vs. AUGMENTED MODEL" + " " * 18)
print("="*80)
print(f"{'Validation Set':<20} | {'Vanilla Model F1':>20} | {'Augmented Model F1
print("-" * 80)

for name in results_vanilla.keys():
    f1_vanilla = results_vanilla[name]['f1_score']
    f1_augmented = results_augmented[name]['f1_score']
    improvement = f1_augmented - f1_vanilla
    print(f"{'name':<20} | {f1_vanilla:>19.2%} | {f1_augmented:>21.2%} | {improvement

print("="*80)

```


FINAL COMPARISON: VANILLA vs. AUGMENTED MODEL			
Validation Set	Vanilla Model F1	Augmented Model F1	Improvement
Original	58.25%	48.59%	-9.6
6% Gaussian Noise	56.86%	53.41%	-3.4
5% Blur	32.59%	34.21%	+1.6
2% Increased Lighting	56.04%	52.76%	-3.2
8%			

Próba poprawek przyniosła tylko lekką poprawę w tylko jednej kategorii - blur, w pozostałych skomplikowanie danych treningowych wpłynęło negatywnie na wyniki.